Internal Software Plan

Implementation Roadmap for the Intersmart Employee Portal

Prepared by Team QA. Based on the manager's sheet *Internal software plan.xlsx* and a review of the current portal code (October 2026).

The short version

• The sheet describes **10 user types** (Sales, Accounts, PMO, PC, UI TL, UX/Dev TL, resources, QA, HR) and about **100 features**.

• About **10 percent already exists** (leave, HR complaints, Hubstaff time, projects, tasks, bugs, recognitions, roles).

• About **45 percent is partly there** and needs extending (project list, tracker, corrections, dashboards).

• About **45 percent is new**: Sales/CRM, Accounts/payments, budget per department, sprints with approval, ratings, feedback, meetings, AMC.

• Estimated effort: **about 27 weeks** with 2 developers, delivered in 8 phases. Each phase is usable on its own.

• **Capacity question:** 100+ users at the same time will **not break the database**. The real risks are hosting limits, constant background polling, and a few security gaps. All are fixable in Phase 0 (about 2 weeks).

Status (grouped features)	Count	Share
HAVE - already works	7	10%
PARTIAL - exists, needs extending	29	43%
BUILD - new	30	45%

Counts are for the grouped feature lines in this document (66 lines), not every sheet row.

How the whole system fits together

Everything in the sheet is one pipeline. Understanding it makes the build order obvious.

1. Sales	2. Accounts	3. PMO	4. Project Coordinator	5. TLs	6. Resources + QA	7. Go live
Lead becomes a project	Advance payment approved	Assigns PC, sets budget per dept	Accepts, fills scope, plans pages and sprints	Approve effort, allocate people	Do daily work, log bugs	System admin deploys; AMC + feedback

Three ideas repeat everywhere. Build them once, reuse everywhere:

• **Approval engine.** Out-of-scope, effort, sprint, budget increase, new page, correction, meeting, appreciation: all are 'someone asks, someone approves or rejects with a reason'. One generic table and screen, not eight.

• **Payment gate.** Project tabs unlock only after payment is approved. Super Admin can override payment; PMO, tech head or HR can override budget. Needs a payment status on every project.

• **Budget tracking.** Budget per department, effort used per task, alert when exceeded, notify PM / management / HR. This feeds ratings and dashboards.

What the portal already gives us

• Login, roles and custom per-user and per-team permissions (Spatie plus custom tables).

• Employees, teams, leave (CL/SL, carry forward, TL then admin approval), WFH, attendance, biometric sync.

• Projects, sub-phases, tasks, task catalog, assignees, comments, attachments, corrections, bug tracker (BugSmart).

• Hubstaff integration (time and activity), recognitions, issues/complaints, announcements, chat, notifications, audit logs.

What we have vs what to build, by role

HAVE = works today. **PARTIAL** = base exists, needs extending. **BUILD** = new. Status is judged from the code (models, routes, pages). Items marked PARTIAL should be confirmed with a quick demo before final estimates.

1. Sales

Feature (from sheet)	Status	What is needed
Create project (same as sales doc)	BUILD	No sales module. Needs a project-intake form with the sales doc fields.
Lead list / follow-up sheet	BUILD	No leads or CRM tables exist.
Inbox (follow-up, proposal, meeting, other)	BUILD	Notifications exist; a categorised, status-based inbox does not.
Payment-approved notification (from Accounts)	PARTIAL	Notification system exists. Payment event must be built first.
Projects list: ongoing/completed + % progress	PARTIAL	Project list + progress exist; needs sales-only view (own projects).
Hold / terminate project	PARTIAL	Project status has On Hold / Cancelled. Needs reason + approval trail.
Out-of-scope request, 'action required', payment follow-up	BUILD	New request types + status reply flow.
Completed status: free support, AMC, next follow-up, feedback	BUILD	New fields and reminders.
Customer feedback / comments any time	PARTIAL	Task comments exist; customer feedback does not.
Leave request, upcoming leaves, HR complaints	HAVE	Leaves, calendar, Issues module already work for all users.
Meeting update / upcoming meetings	BUILD	Calendar has holidays only; no meeting booking.
Sales dashboard (leads, conversion %, target, proposals...)	BUILD	Needs lead + target data first.

2. Accounts

Feature (from sheet)	Status	What is needed
Total project list by payment state (advance awaiting, pending, ...)	BUILD	No payment tables. Core of the payment gate.
Approve payment (notify Sales, assign PMO / notify PMO)	BUILD	Needs payment ledger + notification rules.
Terminate / hold project	PARTIAL	Status exists; permission for Accounts to be added.
Budget increase	BUILD	No budget model (only a single 'allotted_effort' on project).
Set target (sales target)	BUILD	New table: target per person per month.
AMC list	BUILD	New AMC records + renewal reminders.

3. Project Manager (PMO)

Feature (from sheet)	Status	What is needed
Project list + progress + hold reasons (approval, content, SMTP...)	PARTIAL	Project list exists; reason codes and date/budget-exceeded flags are new.
PC-wise project list + PC rating	BUILD	Rating system does not exist.
Assign PC	HAVE	Projects already have a project_coordinator.
Allocate budget per department	BUILD	Core new feature: budget per dept, per project.
Hold project	PARTIAL	Status exists; needs reason.
Approve out-of-scope (to Sales, with effort)	BUILD	Approval engine needed.
Payment request to Accounts / budget allocated view	BUILD	Depends on Accounts module.
Approve corrections with budget	PARTIAL	ProjectCorrection exists; budget check + approval step is new.
Approve/reject meeting request with date + reason	BUILD	Meetings module.
Tracker view: dept-wise, drag-drop, budget left, bugs vs changes	PARTIAL	Task tracker + bug/correction data exist; budget-left and drag-drop layout are new.
Out-of-budget list and action (notify management + HR)	BUILD	Needs budget + alert rules.
Sprint approval from PC	PARTIAL	Task has a 'sprint' text field only; real sprint entity is new.
Monthly customer feedback (mandatory) with rating	BUILD	Question set must be defined by business.
TL / resource feedback, appreciation (group, mail, LinkedIn)	PARTIAL	Recognitions module exists; feedback forms and mail/LinkedIn share are new.
PM dashboard (charts, PC rating, budget chart, Google reviews)	BUILD	Charts on new data. Google review count needs manual entry or API.

4. Project Coordinator (PC)

Feature (from sheet)	Status	What is needed
Accept new project (shows fixed info: platform, pages, budget, quality, salesperson)	BUILD	Needs intake fields + accept step.
Project list with status reasons	PARTIAL	Same as PMO list, filtered to own projects.
Project view with tabs (Scope, Resources, BA, UI, UX, Dev, QA, Bugs, Budget, SysAdmin)	PARTIAL	Project detail page exists; tabs, per-tab payment lock and override rules are new.
Project categories (CMS, Ecom, Custom web, Mobile + tech)	PARTIAL	'category' field exists; the full tree needs seeding.
Business analysis form	BUILD	Purpose, company, competitors, audience, USP, conclusion.
Resources (brand book, logo, domain, brochure...)	BUILD	File/link store per project (storage exists).
UI page definition, sections, budget auto-calc	BUILD	Pages and sections tables + budget formula.
UI daily actions (status chain, Figma link, revisions 1-3)	PARTIAL	Task status exists; page-level status chain is new.
UX / Dev effort request, sprint create, PM approval, date request	BUILD	Effort + sprint approval workflow.
QA auto-allocation on completion, bug list, TL verify	PARTIAL	Bug tracker (BugSmart) exists; auto-handoff is new.
System admin: allocate server, go live	PARTIAL	is_live + live_date already exist; server allocation is new.
PC dashboard	BUILD	Charts on new data.

5. UI TL

Feature (from sheet)	Status	What is needed
New project: allocate resource, schedule, reject with reason	PARTIAL	Task assignees exist; reject-with-reason + scheduling are new.
New task on existing project (correction, extra page, bug...)	PARTIAL	Corrections and tasks exist; scenario types to add.
Resource inbox with progress bars, drag-drop priority	PARTIAL	My Tasks exists; drag-drop priority and expanders are new.
Hubstaff time met / left, activity	HAVE	Hubstaff integration is built and fixed.
Project list / project view with colour for date overrun	PARTIAL	Lists exist; overrun colours + page counts (12/15) are new.
Team list (time, leaves, appreciation, rating, overdue, rejections)	PARTIAL	Team, leave and Hubstaff data exist; rating + rejections new.
Individual feedback (fixed questions)	BUILD	New feedback form.
Appreciation, leave list and approval	HAVE	Recognitions and TL leave approval exist.
TL dashboard	PARTIAL	Dashboard exists; needs TL widgets.

6. UX / Dev TL

Feature (from sheet)	Status	What is needed
Create sprint, reject with reason, sprint approval	BUILD	Sprint entity + approval flow.
Pending tasks: effort sheet, sprint plan, change request, bug review	BUILD	Aggregated work queue.
Resource inbox, project list/view, team list, feedback, appreciation, leave	PARTIAL	Same as UI TL.

7. Resources (UI / UX / Dev / QA)

Feature (from sheet)	Status	What is needed
Profile with rating + metrics	BUILD	Rating engine from rejections, on-time, feedback.
Inbox: projects, expand tasks, start/end task, comment	PARTIAL	My Tasks + comments exist; start/end timers and reasons are new.
Budget-exceeded and date-exceeded reasons	PARTIAL	'deviation_reason' exists on task; make it mandatory and structured.
QA: list bugs with priority flag	HAVE	BugSmart bug tracker covers this.
Apply leave / view leave / HR complaints	HAVE	Already built.
Personal dashboard	PARTIAL	Hubstaff + leave data exist; task chart by budget is new.

8. HR

Feature (from sheet)	Status	What is needed
Leave approval after TL approval	HAVE	Two-step leave approval exists.
Performance dept-wise / resource-wise	PARTIAL	Hubstaff, attendance, recognitions exist; complaints, PC complaints and rejection counts need linking.
Appreciation approval / rejection	PARTIAL	Recognitions exist (Super Admin only); add HR approval.

Build plan: 8 phases

Estimates assume 2 developers (1 backend, 1 frontend) plus QA testing in parallel. They are planning numbers, not promises.

#	Phase	Time	What is delivered
0	Foundation + scale hardening	2 wks	New roles (Sales, Accounts, PMO, PC, TLs, QA). Generic approval engine. Payment-status field on project. Caching, rate limits, token expiry, load-test setup (see capacity section).
1	Sales module	4 wks	Leads and follow-ups, project intake form, sales inbox, meetings request, sales dashboard. Needs the sales doc fields from the manager.
2	Accounts module	3 wks	Payment ledger, advance/pending approval, notify Sales/PMO, budget increase, targets, AMC list.
3	Project core (PC + PMO)	5 wks	Accept project, tabbed project view with payment lock and overrides, categories, BA form, resources, budget per department, PC assignment, out-of-scope flow.
4	UI / UX / Dev workflow	5 wks	Pages and sections with auto-budget, page status chain, effort request, sprints, TL approval, QA auto-handoff, corrections with budget approval.
5	TL and resource screens	3 wks	Inbox with progress and drag-drop priority, start/end task, exceeded reasons, team list, feedback forms, ratings.
6	Dashboards + feedback	3 wks	Dashboards for every role, monthly customer feedback, appreciation approval, Google review count.
7	HR performance, hardening, go-live	2 wks	HR performance views, full QA pass, load test at 150 users, data migration, training.

Total: about 27 weeks. Phases 1 and 2 can run partly in parallel if a third developer joins, which would bring it to about 22 weeks.

Suggested order and why

• Phase 0 first: new features multiply traffic and data; fix the base before piling on.

• Accounts before Projects: the payment gate depends on it.

• Dashboards last: they only look good once real data exists.

• Release after each phase to a small pilot group, so feedback arrives early.

Risks to the schedule

• Sheet is a feature list, not a spec. Many screens need field-level detail (sales doc, feedback questions, rating formula).

• Budget formula (sections x animation level) is not defined yet.

• Rating formula for resources and PCs is not defined; ratings affect people, so HR must sign off.

• Changing how teams already work (Bugzilla-style tracker versus new flow) needs training time.

Can the portal handle 100+ users at once?

Short answer: yes for the database, not yet safely for everything else. The live MySQL 8 database (on a 24-core, ~40 GB RAM server) easily handles hundreds of simultaneous users on a system like this. What can hurt is how the app uses it, and where the backend is hosted. Nothing below is an emergency today; all of it matters before the user count and data volume grow.

What I found in the code

Area	Risk	Finding	Fix
Cache, queue and sessions all use the database	MEDIUM	Live .env sets CACHE_STORE, QUEUE_CONNECTION and SESSION_DRIVER to database. Every page load can read and write session and cache rows in MySQL.	Move cache and sessions to files (or Redis if available). Keep queue in DB only while volume is low.
Constant background polling	MEDIUM	Chat checks every 20s, notifications every 30s, plus tickers every 5 min. 100 users = roughly 600-800 requests/min just idle. Fine now; 300 users doubles it.	Cache unread counts, lengthen intervals when tab hidden, later use push (websocket / SSE).
PHP and MySQL share one server	MEDIUM	Live MySQL runs on the same machine as PHP (127.0.0.1). The machine is strong today (24 cores, ~40 GB), but big reports and dashboards compete with normal page requests.	Index filter columns, cache dashboards, watch the slow-query log. Split the DB to its own server only if load tests show it is needed.
Only one rate limit in the whole API	HIGH	Only the forgot-password route is throttled. Login and file uploads can be hammered.	Add throttle on login (5/min), uploads, and a general API limit.
API tokens never expire	MEDIUM	Sanctum 'expiration' is null. A stolen token works forever.	Set expiry (e.g. 7-30 days) and revoke on password change.
No object-level policies	HIGH for new modules	Access is controlled by role on routes (0 Policy classes). New roles (Sales sees only own leads, PC only own projects) need per-record checks or data leaks across teams.	Add Laravel Policies for every new model before building screens.
MySQL settings and per-account limits	MEDIUM	cPanel can cap connections and processes per account. Defaults for max_connections and the InnoDB buffer pool may be too small for 100+ users plus background jobs.	Check max_connections, innodb_buffer_pool_size, slow-query log; raise if needed.
Heavy dashboard queries	MEDIUM	Dashboards will aggregate across projects, tasks, budgets. Caching is used in only a few places.	Add DB indexes on filter columns, cache dashboards 1-5 min, pre-compute nightly totals.
Live runs in debug mode	HIGH	Live .env has APP_ENV=local and APP_DEBUG=true. Any error page can expose file paths, queries and settings to visitors. Stray files with a hard-coded password also sit in the live repo folder.	Set APP_ENV=production and APP_DEBUG=false, then clear and re-cache config. Delete the stray files after checking them. Rotate the SSH password shared in chat.
Thin automated testing	MEDIUM	Only 14 test files cover a portal of this size. New approval and payment logic needs tests.	Add tests for the approval engine and payment gate before release.
Two copies of the app on the server	MEDIUM	The real live app is ~/public_html/api/Inter-Smart-Portal (branch main, DB workplaceintersm_intersmart_portal, 92 tables). An old copy in ~/public_html/backend (branch office) has a broken DB login and edited files.	Archive or remove the old copy so nobody deploys to the wrong folder. Document the single deploy path.

Good news already in place: 18 list endpoints use pagination, 200+ foreign keys / indexes in migrations, Sanctum token auth, role middleware, audit logs, scheduled jobs with overlap protection.

Capacity plan (do in Phase 0, repeat in Phase 7)

• **Load test** with k6 or Artillery: 50, 100, 150 simulated users doing login, dashboard, task list, check-in. Target: 95 percent of requests under 1 second, no errors.

• **Check MySQL setup:** max_connections, buffer pool size, slow-query log, automatic daily backups (cPanel backup or mysqldump cron) and one tested restore.

• **Security pass:** rate limits, token expiry, policies, file-upload type and size limits, HTTPS only, CORS restricted to the portal domain, dependency update check.

• **Monitoring:** slow-query log, error alerts, uptime check, so problems are seen before users report them.

• **Backups:** test a restore once. A backup never restored is a guess.

Rule of thumb for the manager: with the fixes above, this stack should comfortably serve 150-300 simultaneous users. Beyond that the first step is a bigger backend host, not a new database.

Decisions we need from the manager

• Who are Tech Head, Management and System Admin in the portal? New roles or existing people with permissions?

• Send the **sales doc**: exact fields for project creation and lead tracking.

• Budget rules: how is budget per department calculated (sections x animation level)? Who can change it?

• Rating formula for PC and resources: which inputs, what weights, who can see it?

• Customer feedback questions (response time, clarity, design, bugs). Is it entered by PC or sent to the customer by email link?

• Meetings: only a request/approve list, or also calendar invites and reminders?

• Appreciation sharing to mail / LinkedIn: manual copy or automatic posting? (Automatic LinkedIn posting needs an API approval and is better left manual.)

• Google review count: manual entry or API? (API needs business account access.)

• Current Bugzilla-style tracker versus the new bug flow: replace, or keep both?

• Go-live approach: pilot with one team first, or all at once?

Next step

On approval, start Phase 0 (foundation and scale hardening) and in parallel collect the missing details above. A detailed screen-by-screen spec for Phase 1 and 2 can follow within a week.